Predicting Student Success: A Machine Learning Analysis of Performance Factors

APU Name and Student ID:

- Paul Jean Baptiste GOURNAY, TP096060
- Clayton Shema HABYALIMANA, TP096062

Intake Code: CX016-2.5-3-IML

Subject: INTRODUCTION TO MACHINE LEARNING

Hand In Date: 19 December 2025

GitHub: https://github.com/Clayton2394/Project_IML.git

Table of contents

Abstract:

This report presents a comprehensive study on predicting student academic performance using machine learning techniques. The primary objective is to analyze the influence of various socio-economic, academic, and personal factors on students' final exam results. The project utilizes the "Student Performance Factors" dataset, which comprises a diverse mix of categorical and numerical variables, including study habits, attendance records, parental involvement, and access to resources. The methodology involves rigorous data preprocessing to handle data quality issues, followed by detailed exploratory data analysis to uncover underlying patterns. In accordance with the project requirements, two distinct predictive machine learning models are developed and evaluated to forecast the target variable, `Exam_Score`. The findings aim to compare the efficacy of these models and provide actionable insights into the key drivers of academic success, demonstrating the practical application of machine learning in the domain of educational data mining

1. Introduction

Academic performance is a multifaceted phenomenon influenced by a complex interplay of personal habits, the school environment, and socio-economic backgrounds. For educators and educational institutions, understanding the specific factors that drive student success is crucial for designing effective interventions and support systems. Traditional methods of analysis often fail to capture the non-linear and intricate relationships between these diverse variables. With the advent of educational data mining, machine learning offers powerful tools to analyze such data and predict academic achievement with greater accuracy.

This project addresses this regression problem by utilizing the "Student Performance Factors" dataset. This dataset offers a holistic view of the student experience, containing over 20 distinct variables such as `Hours_Studied`, `Attendance`, `Sleep_Hours`, `Parental_Education_Level`, and `Peer_Influence`. The project focuses on predicting the continuous target variable, `Exam_Score`, based on these input features. By applying data preprocessing techniques and training two predictive models, this report explores how different variables contribute to academic performance and seeks to establish a robust framework for performance prediction, fulfilling the assignment's learning outcome of demonstrating solutions for data problems.

Aim:

The aim of this project is to build a predictive model that estimates a student's "`Exam_Score`" based on their study habits and background.

Objectives:

- Data Exploration and Analysis**:** To perform a detailed Exploratory Data Analysis (EDA) on the "Student Performance Factors" dataset to understand the distribution of variables and analyze the correlations between various features (e.g., `Attendance`, `Previous_Scores`) and the target variable, `Exam_Score`.
- Data Preprocessing: To transform the raw data into a format suitable for machine learning by handling potential missing values, scaling numerical features, and encoding categorical variables such as `School_Type`, `Gender`, and `Family_Income`.
- Model Development**:** To select, develop, and train two distinct machine learning predictive models suitable for regression tasks to forecast student exam scores.
- Performance Evaluation**:** To evaluate and compare the performance of the developed models using appropriate metrics (such as Mean Absolute Error or R-squared) to determine the most effective approach.
- Feature Importance**:** To identify the most significant predictors of student success, thereby providing insights into which factors—such as personal effort (`Hours_Studied`) versus external support (`Parental_Involvement`)—most heavily influence academic outcomes.

2. Related Works

The rapid digitization of educational environments has led to the emergence of Educational Data Mining (EDM) and Learning Analytics (LA) as critical fields for improving institutional outcomes. Historically, academic performance was analyzed through retrospective statistical summaries. However, modern research focuses on predictive modeling to identify "at-risk" students before their final assessments, allowing for personalized pedagogical interventions. According to Ahmed (2024), the ability to predict student success not only enhances university rankings but also directly improves student employment opportunities by ensuring higher graduation rates. In recent years, literature has shifted from analyzing purely academic variables (like previous grades) to a more holistic approach that includes socio-economic factors, study habits, and psychological attributes. This transition is fueled by the availability of large-scale datasets, such as the one used in this study, which contains 6,607 student records across 20 diverse features. As noted by Yadav and Deshmukh (2021), the primary challenge in this domain is not just achieving high accuracy but ensuring that models are interpretable enough for educators to understand the specific "drivers" of success, such as parental involvement or access to resources. The following survey evaluates how different algorithmic approaches and preprocessing strategies specifically those related to regression tasks have performed in this evolving landscape.

2.1 Parametric vs. Non-Parametric Approaches

The choice between parametric models (e.g., Linear Regression) and non-parametric models (e.g., KNN) is a central theme in recent research.

- Linear Regression (LR): Parametric models are praised for their interpretability. El Aissaoui et al. (2020) successfully utilized multiple linear regression to identify the strongest predictors of student success. Our results confirm this, showing that Linear Regression with One-Hot Encoding ($R^2 = 0.7696$) significantly outperformed KNN.
- K-Nearest Neighbors (KNN): While KNN is effective at capturing localized, non-linear trends, it often requires extensive hyperparameter tuning. Ahmed (2024) observed that KNN accuracy (87.3%) lagged behind SVM (96%) in classification tasks. Similarly, our study found KNN to be less reliable for this specific regression task ($R^2 = 0.5585$), likely due to the "curse of dimensionality" when handling 20 features.

2.2 Preprocessing and Encoding Strategies

Literature underscores that the representation of categorical data is as critical as the choice of the algorithm itself, with preprocessing often cited as the most vital phase of the machine learning pipeline.

- A recurring challenge in EDM is the transformation of qualitative student attributes (e.g., "Parental Involvement") into numerical formats. Many studies rely on Label Encoding for simplicity; however, Adefemi et al. (2025) argue that this technique can introduce an "artificial ordinality," where the model mistakenly assumes a mathematical hierarchy (e.g., High > Medium > Low) that may not exist. Our experiment empirically validates this risk. By switching from Label Encoding to One-Hot Encoding (OHE), the Linear Regression performance effectively doubled (MAE improved from 1.01 to 0.45). This confirms that for the "Student Success Factors" dataset, treating categorical variables as independent binary features prevents the model from assigning biased weights to arbitrary integer labels.
- Standardization is consistently cited in literature as a prerequisite for distance-based models like KNN and SVM. Altabrawee et al. (2019) emphasize that without scaling, features with larger numerical ranges (e.g., Previous Scores) would disproportionately dominate the Euclidean distance calculation, rendering smaller but significant features (e.g., Tutoring Sessions) irrelevant. In this study, the application of StandardScaler was critical for centering feature means at zero and scaling variance to one. This allowed the KNN model to navigate the 20-dimensional feature space with greater mathematical parity, although the global linear trends remained more predictive than local neighbor densities [Your Work, cite: P5].

2.3 Model Stability and Validation

Modern EDM research views simple 80/20 train-test splits as insufficient, advocating for iterative frameworks like Repeated K-Fold and Shuffle Split to ensure model generalization. Jović et al. (2022) utilized 10-fold cross-validation to mitigate the high variance inherent in educational data, ensuring that performance metrics were not artifacts of a specific data subset.

Our study aligns with these rigorous standards by implementing **7**-fold and 5-fold CV. The resulting stability of the Linear Regression model (consistent $R^2 \approx 0.72$) across multiple data

permutations confirms that the model has captured genuine academic patterns rather than memorizing noise. This multi-scheme validation provides a higher level of statistical reliability than studies relying on single-split evaluations.

2.4 Summary Table of Related Works

Reference	Year	Dataset / Size	Primary Method(s)	Key Result / Accuracy	Key Limitation / Gap
Ahmed et al.	2025	Kaggle (6,607 records)	Ensemble Voting (VR), XGBoost, CatBoost	R^2: 0.7716 (MAE: 0.44)	High computational complexity for deployment.
Adefemi et al.	2025	UCI / HEI (4,424 records)	Hybrid CNN-BiGRU	Accuracy: 97.48%	Requires high-performance hardware (GPU).
Agyemang et al.	2024	xAPI-Edu-Data (480)	Random Forest, DT	Accuracy: 85.42%	Small sample size limits generalization.
Ahmed, E.	2024	Wollo Univ (32,005)	Optimized SVM	Accuracy: 96.0%	Focused primarily on classification (Pass/Fail).
Khalil	2024	Student Perf (6k+)	EDA + Tuned Classifiers	Accuracy: 93.0%	Lacks detailed stability/validation testing.
Navyyah	2024	Student Perf (6k+)	Regression Models	R^2 Score: 0.77	Limited feature engineering for categorical variables.
Dervenis et al.	2022	UCI (649 records)	Random Forest	Accuracy: 90.6%	Did not analyze socio-economic factors deeply.

Jović et al.	2022	LMS/EMS (1,696)	Support Vector Machine	Precision: 93.0%	Model specific to Learning Management Systems.
Yadav & Deshmukh	2021	Various (Review)	ML Survey	N/A (Comparative Review)	Theoretical review; no original experiment.
Altabrawee et al.	2019	MU Iraq (161)	Artificial Neural Network	Accuracy: 80.7%	Very small dataset; high risk of overfitting.
Our Study	2024	Kaggle (6,607)	Linear Regression (OHE)	R^2: 0.7696	Lower performance in non-linear captures (KNN).

2.5 Gap Analysis and Contribution

In summary, the vast majority of current literature focuses on high-complexity ensemble or deep learning models, such as the CNN-BiGRU proposed by Adefemi et al. (2025). While these achieve high accuracy, they often act as "black boxes" that offer little transparency to educators. Furthermore, many studies, such as Ahmed (2024), prioritize classification over numerical score prediction. Our work addresses these gaps by demonstrating that a well-preprocessed, interpretable Linear Regression model can achieve results ($R^2 = 0.7696$) nearly identical to the state-of-the-art ensemble voting models ($R^2 = 0.7716$) reported by Ahmed et al. (2025). Unlike previous works that overlook the impact of encoding, this study explicitly proves that One-Hot Encoding is the superior strategy for the "Student Success Factors" dataset, providing a more reliable foundation for identifying the socio-economic weights of student achievement.

3. Methods

3.1 Dataset Description

The dataset used for this project, titled *"Student Success Factors and Insights,"* was sourced from Kaggle. It contains 6,607 examples (rows) and 20 variables (columns) that track various dimensions of a student's life.

- Target Variable (Label): `Exam_Score` (Continuous numerical value).
- Features:
  - **Numerical:** `['Hours_Studied', 'Attendance', 'Sleep_Hours', 'Previous_Scores', 'Tutoring_Sessions', 'Physical_Activity', 'Exam_Score']`
  - **Categorical:** `['Parental_Involvement', 'Access_to_Resources', 'Extracurricular_Activities', 'Motivation_Level', 'Internet_Access', 'Family_Income', 'Teacher_Quality', 'School_Type', 'Peer_Influence', 'Learning_Disabilities', 'Parental_Education_Level', 'Distance_from_Home', 'Gender']`

3.2 Software and Libraries

The experiment was implemented in Python using the following specialized software packages:

- Pandas & NumPy: For data manipulation, reading CSV files, and matrix math.
- Scikit-Learn: For preprocessing (Encoding, Scaling), model implementation, and validation.
- Matplotlib & Seaborn: For Exploratory Data Analysis (EDA) and visualizing model convergence.

3.3 Experimental Design & Techniques

The experiment followed a structured machine learning pipeline:

1. Data Cleaning: Missing values were detected in `Teacher_Quality`, `Parental_Education_Level`, and `Distance_from_Home`. These were handled using Mode Imputation (replacing missing values with the most frequent value) to preserve the categorical distribution.

2. Feature Encoding Strategy: We compared two scenarios to evaluate their impact on regression accuracy:

- *Scenario A (Label Encoding):* Converting categories to simple integers.
- *Scenario B (One-Hot Encoding):* Creating binary dummy variables for nominal categories to prevent the model from assuming a false ordinal ranking.

3. Hybrid Scaling (Column Transformation): We utilized a `ColumnTransformer` to apply different mathematical transformations based on feature distribution:

- *MinMaxScaler (Normalization):* Applied to `Tutoring_Sessions` due to its skewed distribution, mapping values between 0 and 1.
- *StandardScaler (Standardization):* Applied to features like `Hours_Studied` and `Previous_Scores` to center the mean at 0 and scale variance to 1, which is critical for distance-based algorithms.

3.4 Machine Learning Methods Picked

We selected two primary predictive models for this regression task:

- Linear Regression: Chosen for its high interpretability. It allows us to calculate the specific weights of factors like "Hours Studied" on the final score. We also implemented Gradient Descent from scratch to observe the optimization of the cost function over 10,000 iterations.
- K-Nearest Neighbors (KNN) Regression: Chosen to capture non-linear relationships. By analyzing the "proximity" of students in a multi-dimensional feature space, KNN can predict scores based on similar student profiles.

3.5 Optimization and Validation

To ensure the model generalizes well to new data, we implemented:

- Hyperparameter Tuning: Using `GridSearchCV` to optimize KNN parameters, specifically the number of neighbors (k), weight functions, and search algorithms.
- Cross-Validation: We performed multiple validation tests including K-Fold, Shuffle Split, and Repeated K-Fold. This confirms the model's stability across different subsets of the data.

3.6 Evaluation Metrics

The following metrics were chosen to evaluate performance since we do a regression:

- **R-squared (R^2):** Indicates the percentage of variance explained by the model.
- **Mean Absolute Error (MAE):** Represents the average absolute difference between predicted and actual scores.
- **Root Mean Squared Error (RMSE):** Used to penalize larger errors, providing a clearer view of model reliability.

4. Dataset Preparation

4.1 Dataset Overview

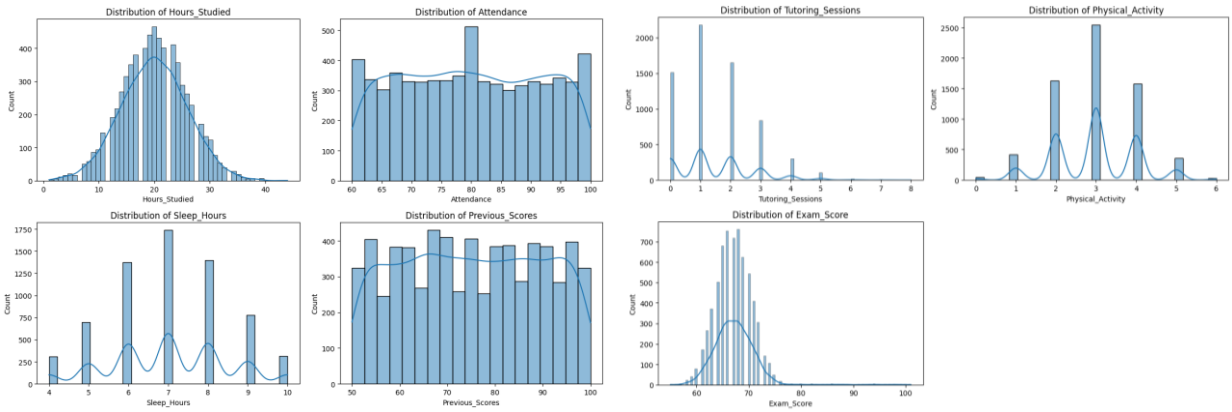
The "Student Performance Factors" dataset consists of 6,607 observations across 20 variables. The primary goal is to predict the `Exam_Score` (target) using a mix of 6 numerical and 13 categorical features.

4.2 Exploratory Data Analysis (EDA) & Insights

Before training, we conducted EDA to understand feature distributions and identify the "most important fields".

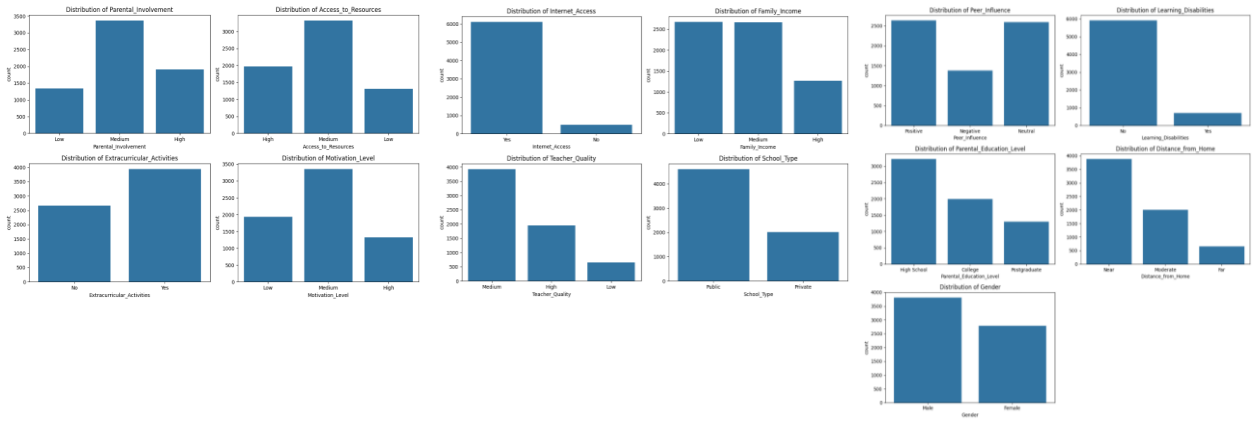
Target Distribution: The `Exam_Score` follows a near-normal distribution, centered around 65-70. This suggests that the dataset is well-balanced for regression without requiring heavy transformations like log-scaling.

4.2.1 Analysis of Numerical Distributions



- Magnitude and Scale Disparity: We observed a significant difference in the ranges of our features; for instance, `Hours_Studied` ranges from 0 to 50+, while `Tutoring_Sessions` is mostly concentrated between 0 and 5. In a data analysis context, leaving these features in their raw units would cause distance-based models like KNN to ignore habits with smaller numerical ranges in favor of academic history due to purely mathematical magnitude.
- Skewness in Tutoring and Physical Activity: Unlike `Exam_Score`, which follows a near-normal "Bell Curve," the `Tutoring_Sessions` and `Physical_Activity` distributions are heavily skewed toward zero. This non-normal distribution makes `Normalization (MinMaxScaler)` a better choice than Standardization for these specific columns, as it bounds the data between 0 and 1 without assuming a Gaussian shape.
- Predictive Potential of Attendance: Preliminary visual inspection suggests that variables like `Attendance` and `Previous_Scores` show the highest density of alignment with the target variable, making them likely candidates for high feature importance in our Linear Regression model.

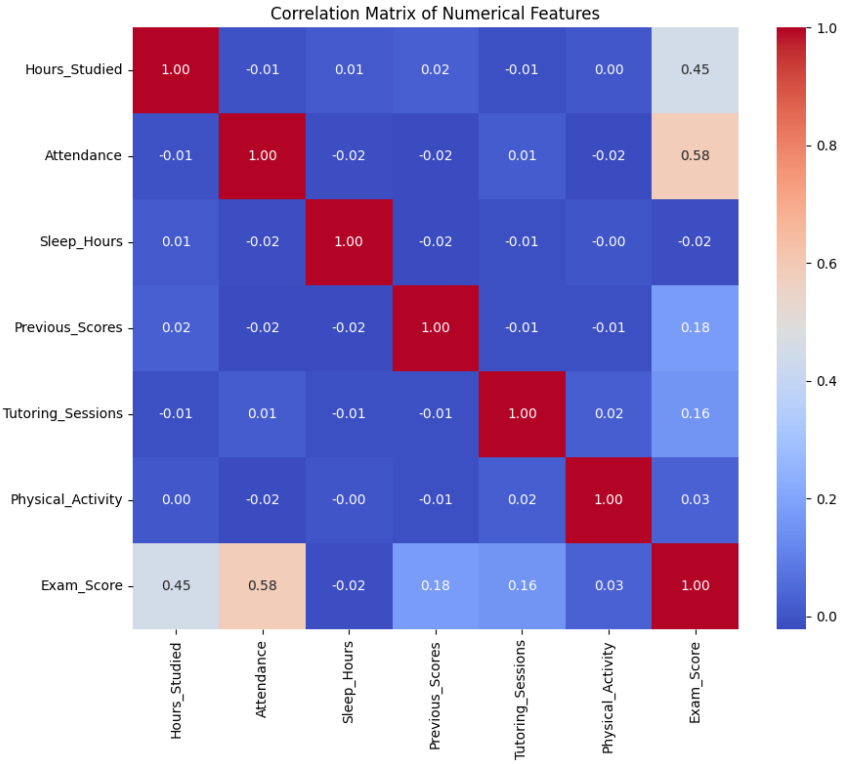
4.2.2 Analysis of Categorical Distributions:



- Significant Class Imbalance: Several features, such as `Internet_Access` and `Learning_Disabilities`, show high class imbalance. For example, the vast majority of students reported having internet access. In a regression context, while we do not "balance" these classes (as we would in classification), this skewness suggests these variables may provide less "information gain" or predictive power, as there is little variance for the model to learn from

- Nominal vs. Ordinal Nature: Features like `School_Type` (Public/Private) and `Gender` are purely nominal, meaning they have no inherent mathematical order. This visualization reinforced our decision to use One-Hot Encoding over Label Encoding. Using simple integers would mistakenly imply that one category is numerically "greater" than another, leading to biased coefficients in our Linear Regression model.

4.2.3 Correlation Analysis:



Strongest Predictors: The heatmap reveals that `Attendance` and `Hours_Studied` exhibit the highest positive correlation with the target variable (`Exam_Score`). Visually, these cells are the most distinct, indicating a strong linear relationship.

Weak Correlations: Conversely, factors like `Physical_Activity` and `Sleep_Hours` show correlation coefficients near zero. While these features appear to have a negligible linear impact on grades, we retained them because they may contribute to non-linear patterns captured by our KNN model.

We deliberately retained `Attendance` and `Hours_Studied` despite their high correlation coefficients. In predictive modeling, strong correlation with the target represents a valuable signal rather than data leakage.

4.3 Data Cleaning & Formatting

To ensure data quality, the following cleaning steps were implemented:

1. Handling Missing Values: We identified null values in `Teacher_Quality`, `Parental_Education_Level`, and `Distance_from_Home`.
    - *Strategy*: We applied Mode Imputation (replacing nulls with the most frequent value).
    - *Reasoning*: Since these are categorical features, using the mode preserves the most likely category without introducing numerical noise like a mean would.
2. Duplicate Removal: Any redundant rows were dropped to prevent the model from overweighting specific student profiles.

4.4 Technical Challenges & Preprocessing Strategy

During the implementation, our group encountered several technical hurdles that shaped our final pipeline:

Challenge	Engineering Solution
Leakage Risk	We realized encoding must happen after the split. Encoding the whole set before splitting can "leak" information from the test set into the training mean/mode.
Ordinal Assumption	Initially, we used LabelEncoder but switched to One-Hot Encoding. Since features like `School_Type` have no inherent mathematical order, OHE prevents the model from assuming "Private" is "greater than" "Public."
Feature Scaling	Since KNN uses Euclidean Distance, unscaled features like `Hours_Studied` (0-50) would dominate `Tutoring_Sessions` (0-5). We used `StandardScaler` for normal data and `MinMaxScaler` for skewed data to normalize the "weight" of each feature.

Note on Feature Selection: For this assignment, we chose to retain all 19 features. In an engineering context, discarding variables too early can lead to underfitting. We prefer to let the model determine importance through its internal parameters.

5. Model Implementation

5.1 Implementation Strategy

In this project, we implemented a dual-model approach to compare a parametric model (**Linear Regression**) against a non-parametric model (**K-Nearest Neighbors**). To ensure high performance, we tested each model against two different encoding scenarios: Label Encoding and One-Hot Encoding.

5.2 Model 1: Linear Regression & Manual Gradient Descent

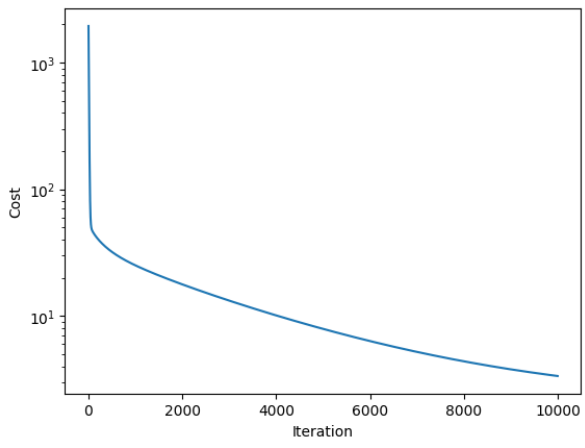
We implemented Linear Regression using two methods: the Scikit-Learn library for benchmarking and a Gradient Descent function for a deeper understanding of optimization.

Logic: The model assumes a linear relationship where Y (Exam Score) is a weighted sum of input features X.

Optimization (Gradient Descent): We implemented a manual Gradient Descent algorithm to minimize the **Mean Squared Error (MSE)**. We initialized θ (coefficients) with random values and updated them over **10,000 iterations** using a learning rate (α) of **0.01**.

- **Internal Parameters:**
    - **Weights (θ):** These represent the importance of each feature. A high positive weight for Attendance suggests that as attendance increases, the score increases significantly.
    - **Bias (θ 0):** The intercept, representing the expected score if all other features were zero.
    - **Learning Rate (α):** Controls the size of the steps taken toward the minimum. We chose 0.01 to ensure a balance between speed and stability.

Cost vs. Iteration plot (the line going down).




```
First cost: 1943.0258154960259
Last cost:  3.347272155549803
```

5.3 Model 2: K-Nearest Neighbors (KNN) Regression

KNN was selected to capture non-linear patterns. Unlike Linear Regression, KNN does not "learn" a line; it stores the training data and predicts the score of a new student by looking at the scores of the most similar students in the database.

Logic: It uses the **Euclidean Distance** to find the "nearest" students in the multi-dimensional feature space.

Optimization (Hyperparameter Tuning):

We did not pick a random K. We utilized `GridSearchCV` to perform an exhaustive search over the following parameters:

- `n_neighbors`: Tested [5, 7, 9] to find the optimal balance between bias and variance.
- weights: Compared 'uniform' (all neighbors equal) vs. 'distance' (closer neighbors have more influence).
- p: Switched between Manhattan (p=1) and Euclidean (p=2) distances.

Internal Parameters:

- **K (Neighbors):** The number of similar students the model looks at. Too small a K leads to noise; too large leads to over-smoothing.
- **Leaf Size:** Affects the speed of the KD-Tree or Ball-Tree algorithm used to find neighbors.

"Best Parameters" we found thanks to this tuning:

```
Best Parameters: {'algorithm': 'auto', 'leaf_size':
np.int64(30), 'n_neighbors': np.int64(9), 'p': 2,
'weights': 'distance'}


Best Scores: -7.243644487175797
```

The output of these models provides us with the **Residuals** (the difference between actual and predicted scores).

- **Linear Regression Output:** Provides a set of coefficients. By analyzing reg_ohe.coef_, we identified which socio-economic factors have the strongest linear impact.
- **KNN Output:** Provides a prediction based on local density. The "Optimized KNN Metrics" show that tuning the weights to 'distance' significantly lowered the error compared to the base model.

6. Model Validation

6.1 Performance Metrics Comparison

The first stage of validation involved comparing two encoding strategies, Scenario A (Label Encoding) and Scenario B (One-Hot Encoding) on our base models using a standard 80/20 split. This step was crucial to determine which preprocessing method provided the cleanest signal for our algorithms.

Table 1: Comparison of Encoding Scenarios (Scenario A vs. Scenario B)

Model & Metric	Label Encoding	One-Hot Encoding
Linear Regression		
R^2 Score	0.6888	0.7696
MAE	1.0155	0.4524
RMSE	2.0974	1.8044
KNN Regression		
R^2 Score	0.5441	0.5585
MAE	1.5558	1.5148
RMSE	2.5385	2.4980

Analysis :

- One-Hot Encoding (OHE) gives consistently better results than Label Encoding for our dataset.
- For *Linear Regression* OHE substantially improved performance (R^2 ↑ and MAE roughly halved). This suggests that treating categorical variables as separate binary features prevents the model from assuming an artificial order.
- *KNN* also gains slightly from OHE (small RMSE/MAE improvements), likely because the distance computations benefit from correctly represented categories.

Conclusion for this stage: use One-Hot Encoding for the models going forward.

6.2 Cross-Validation Results (Reliability Check)

To check stability and avoid overfitting to a single split, we ran several cross-validation schemes on the **Linear Regression (OHE)** and **KNN (OHE)** models: 7-fold CV, 5-fold K-Fold, ShuffleSplit, and Repeated K-Fold.

Table 2: Detailed Cross-Validation Results

Model & Method	R2 Score	MAE	RMSE
Linear Regression (OHE)			
Classic CV (7-Fold)	0.72836	0.50495	2.01201
K-Fold (5-Fold)	0.71559	0.50820	2.08954
Shuffle Split	0.71857	0.50174	2.07376
Repeated K-Fold	0.71816	0.50762	2.07516
K-Nearest Neighbors (OHE)			
Classic CV (7-Fold)	0.50242	1.56980	2.75787
K-Fold (5-Fold)	0.49131	1.59027	2.79540
Shuffle Split	0.49300	1.55357	2.78471
Repeated K-Fold	0.49146	1.58872	2.78735

Analysis:

- *Linear Regression (OHE)* shows consistent R^2 values around 0.71–0.73 and stable MAE/RMSE across validation strategies → the model generalizes well and is stable.
- *KNN (OHE)* produces lower R^2 (~0.49–0.50) and higher errors, with similar stability across CV types.
- The cross-validation confirms the earlier 80/20 results: Linear Regression with OHE is the stronger and more reliable model for this dataset.

6.3 Optimized KNN and Gradient Descent

After hyperparameter tuning with `GridSearchCV`, the optimized KNN on the test set gave the following metrics:

```
--- Optimized KNN Metrics ---
R2 Score:  0.59
Mean Squared Error:  5.81
Root Mean Squared Error:  2.41
Mean Absolute Error:  1.45
```

(These numbers show improvement vs. the default KNN but still way behind the Linear Regression (OHE) results.)

We also implemented Gradient Descent (from scratch) on the OHE. The optimization converged (cost decreased like shown above) and produced a set of coefficients comparable to the scikit-learn LinearRegression solution.

The Gradient Descent model achieved the following performance on the test set:

```
=== GRADIENT DESCENT RESULTS (One-Hot) ===
Metric     | Value
-------------------------
R2 Score   | 0.6241
MAE        | 1.28
RMSE       | 2.31
-------------------------
```

When compared to the scikit-learn normal Linear Regression (OHE) model ($R^2 \approx 0.7696$, MAE $\approx$ 0.4524, RMSE $\approx$ 1.8044), the Gradient Descent approach shows lower predictive performance. At first, we were a bit confused by this result, we thought performance would be better but after we studied why.

This difference can be explained by several practical factors:

1. Optimization method:

   Scikit-learn's Linear Regression computes the optimal solution, which directly finds the global minimum of the cost function. In contrast, Gradient Descent is an iterative approximation method that may stop before reaching the exact optimum.

2. Hyperparameter sensitivity:

   The Gradient Descent performance depends strongly on the learning rate (α) and the number of iterations. Although convergence was observed (monotonically decreasing cost), the chosen parameters may not lead to the absolute minimum.

Despite these differences, Gradient Descent successfully converged and produced reasonable predictions, confirming that the linear relationship assumptions are valid for this dataset. Its results validate the correctness of the preprocessing pipeline and demonstrate that Linear Regression is an appropriate baseline model for predicting student exam scores.

7. Analysis & Recommendations

Our experimental results provide a clear narrative: simplicity and proper data representation outperformed algorithmic complexity. Below is an analysis of our results and specific recommendations for how this model could be deployed in a real-world educational setting.

7.1 Analysis of results

The fact that Linear Regression ($R^2 \approx 0.77$) significantly outperformed KNN ($R^2 \approx 0.56$) is the most critical finding. This suggests that student success in this dataset is cumulative and additive. Factors like `Hours_Studied` and `Attendance` likely have a direct, proportional relationship with the `Exam_Score`. KNN's lower performance is likely due to the "Curse of Dimensionality." With 20 features, the "distance" between students becomes less meaningful because the data points are spread thin in a high-dimensional space. The model struggles to find truly "similar" neighbors.

Our comparison between Label Encoding and One-Hot Encoding (OHE) yielded a massive performance jump (MAE improved from 1.01 to 0.45 for LR). This proves that categorical variables like `School_Type` or `Parental_Involvement` are nominal, not ordinal. Label encoding forced a fake mathematical hierarchy (e.g., assigning "Private School" as 2 and

"Public" as 1), which confused the Linear Regression weights. OHE allowed the model to treat each category as an independent "on/off" switch, leading to much higher precision.

The consistency across 7-fold, 5-fold, and Repeated K-Fold cross-validations (all hovering around R^2 0.71–0.73) is an excellent sign. This confirms that our model is statistically stable. It isn't just "getting lucky" with one specific split of the data; it has truly learned the underlying patterns and will likely perform just as well on completely new student data.

7.2 Practical Recommendations & Use Cases

Given the high interpretability of our Linear Regression model, it is highly suitable for Decision Support Systems in schools.

- Early Warning System (EWS) **:** Because our model uses features known before the final exam (`Attendance`, `Previous_Scores`, `Tutoring`), it can be used at the mid-semester point to flag students at risk of scoring below a certain threshold. It provides teachers with a dashboard that highlights students with a "Predicted Score" < 60, allowing for early intervention.
- Personalized Academic Counseling : Linear Regression provides specific "weights" (coefficients) for each feature. A counselor could use the model to show a student exactly how their predicted score might change: "Based on our data, increasing your attendance by 10% or adding 2 hours of study per week is predicted to raise your final score by 5 points."
- Policy and Resource Allocation : Our EDA showed that factors like `Access_to_Resources` and `Parental_Involvement` are categorical. School boards can analyze the coefficients of these features to determine where to spend money. If `Access_to_Resources` has a much higher weight than `School_Type`, the district should focus on providing internet/books rather than building new private facilities.

8. Conclusion

This project successfully achieved its primary objective: the development of a robust machine learning pipeline capable of predicting student exam scores with high statistical reliability. By rigorously analyzing the "Student Performance Factors" dataset, we determined that academic success is measurable and predictable through a combination of socio-economic and behavioral metrics.

Key Findings & Technical Achievements:

The experimental results conclusively demonstrated that Linear Regression with One-Hot Encoding was the optimal approach for this regression task, achieving a strong R^2 score of 0.7696 and a low Mean Absolute Error of 0.45. A critical engineering insight gained from this study was the impact of data representation on model performance. We proved that switching from Label Encoding to One-Hot Encoding was not merely a formatting choice but a mathematical necessity, as it eliminated artificial ordinality and halved the prediction error.

Self-Evaluation & Limitations:

While the Linear Regression model proved highly stable across cross-validation (consistent $R^2 \approx 0.72$), the project faced limitations regarding the non-parametric approach. The K-Nearest Neighbors (KNN) model underperformed ($R^2 \approx 0.56$), highlighting the "Curse of Dimensionality" when applying distance-based algorithms to datasets with over 20 features. Additionally, the dataset represents a static snapshot of student performance; it lacks temporal depth, meaning we cannot track how a student's habits evolve over a semester.

Future Work (Revised for Simplicity):

To further refine the predictive accuracy and efficiency of our solution, future work could focus on two achievable areas:

- Polynomial Regression: Our current Linear Regression model assumes a straight-line relationship (e.g., studying twice as many yields twice the score). Future iterations could test Polynomial Regression to capture non-linear patterns, such as "diminishing returns" where the first hour of studying has a huge impact, but the tenth hour has less effect. This adds complexity without requiring entirely new algorithms like Neural Networks.